

Linear Programming of Graph Squares

Eddie Xu

Submitted for the degree of Master of Science:
Mathematics and Statistics

Supervisor: Christopher Duffy

August 11, 2026

Contents

1	Introduction	2
1.1	Historical Background	2
1.2	Aims	3
1.3	Literature	3
1.4	Definitions	4
1.4.1	Complexity Theory	4
1.4.2	Graph Theory	5
1.4.3	Mathematical Optimization	5
2	Algorithmic Approaches	7
2.1	Polynomial Time Algorithm	7
2.2	NP-Completeness Proofs	10
2.3	Integer Program Foundation	10
2.3.1	Background	10
2.4	Linear Program Relaxation	11

Chapter 1

Introduction

1.1 Historical Background

In 1949, Leon Festinger proposed a novel method in the field of social psychology for analyzing relationships among a number of people present [FBB23]. Classically this was represented using a sociogram, a visual tool that maps out the relationships between people, however Forsyth and Katz introduced the use of a matrix to represent the sociometric pattern, with rows and columns being individuals and containing 1s for choices and blank diagonals. Festinger noticed that the action of squaring a matrix

$$A_{rc}^2 = A_{1c}A_{r1} + \cdots + A_{nc}A_{rn}$$

represented the number of “two-step” connections, which can then be used to denote “three-step” connections between any two people

$$A_{rc}^3 = A_{1c}A_{r1}^2 + \cdots + A_{nc}A_{rn}^2,$$

such that higher steps can be easily generated. In this manner, higher powers of the adjacency matrix encode the number of walks of increasing length between pairs of individuals, allowing densely connected groups of people, known as cliques, to be identified. In particular, cliques between any three individuals can be done by looking at the diagonals of the cubed matrix A^3 .

This matrix A can be considered as a form of an adjacency matrix, and in the process of improving clique detection methods, Ross and Harary showed that complete graphs may be realized as squares of star trees by considering powers of such matrices [RH60]. They introduced a structural characterization of the square of a graph in terms of clique intersections, “multicliquial” vertices, and so called “neighborly” properties, proving that a graph is the square of a tree if and only if

- i) every vertex is “neighborly” - is not isolated,
- ii) no two cliques intersect in more than two vertices, and
- iii) the number of clique pairs intersecting in two vertices is exactly one less than the total number of cliques.

¹ A_{rc}^2 is more naturally written as $\sum_{k=1}^n A_{ck}A_{kr}$ using the commutativity of multiplication $A_{1c}A_{r1} = A_{c1}A_{1r}$, which intuitively gives the notion of paths from c to r via intermediary k

² A_{rc}^3 is more naturally written as $\sum_{k=1}^n A_{ck}A_{kr}^2$, which intuitively counts the number of paths from c to r that go through an intermediate vertex k , followed by any two-step path from k to r .

Using these properties, they constructed an explicit algorithm that recovers the underlying tree for a given square graph, and their seminal paper has become the foundational reference for the study of graph squares and is universally cited in subsequent literature on the subject.

Since then, the problem of finding the square root of a graph has extended beyond sociology and is a well-studied field in both mathematics and computer-science, with direct applications in distributed computing (e.g., see [PA23]), network design and computational biology. This is in part because many structural properties of graph roots are first established in the square case before being extended to the more general k -th power setting (e.g., see [HKT67]).

Research in this area has primarily focused around an undirected graph H and its square $G = H^2$ which is obtained by adding an edge between every pair of vertices whose distance in H is at most two. This two-step undirected graph square has been studied extensively with much of the focus being on the complexity of the decision problem - given a specific square graph G does there exist a root graph H such that $H^2 = G$. Motwani and Sudan showed that graph squares are NP-complete in general and hence subsequent studies in the area have restricted the decision problem in two ways, either by considering various properties on the root graph or by considering properties on the square graph.

1.2 Aims

In contrast, rather than studying the construction problem directly, in this work we analyze the relationship between the adjacency matrix A_H of the root and the adjacency matrix A_G of the square, and show that the problem can be completely categorized as an integer program. We then examine the relaxed linear program which can be evaluated in polynomial time using interior point method, and integrate existing literature on special root structures as a stepping-stone towards generating a true polynomial time algorithm.

We begin by introducing the comprehensive literature surrounding the subject and provide a general overview of the research direction in this area and the major advances that have been made over the years. We then standardize the definitions that will be used for the rest of this thesis and introduce fundamental theory on the three areas of complexity theory, graph theory and mathematical optimization that are used in the various algorithmic approaches to the decision problem.

The first algorithm we explore is the polynomial time algorithm given by Ross and Harary for constructing a tree root from a given square. We then explore the proof used by Motwani and Sudan in showing that

1.3 Literature

Focusing first on special undirected graph classes, the aforementioned work by Ross and Harary in 1960 gives an algorithm for constructing a tree root by using characterizations of the square graph. As part of this work they proved tree square roots are unique up to isomorphism. This was the first major contribution to the study of graph roots and provided fundamental structural characteristics of square graphs, laying the groundwork for later research on graph root problems and their applications [RH60].

Fleischner in 1974 showed that as long as a graph is 2-connected its square graph will have a Hamiltonian cycle. This established a broad condition guaranteeing Hamiltonicity in graph squares which is used in many subsequent works [Fle74].

Lau and Corneil in 2004 provided an algorithm for constructing square roots from an interval square graph in polynomial time, and showed that the decision problem for finding roots of chordal graphs, roots of split graphs, and chordal roots of graphs are NP-complete [LC04]. Furthermore, Lau considered the construction of square roots from a bipartite graph in linear time, and showed that the existence of cube roots in bipartite square is NP-complete [Lau06].

Le and Tuy in 2007 generalized tree-root results further by giving a linear time recognition of roots of block graphs [LT07].

Extending to the broader generalizing of graph squares, Mukhopadhyay in 1967 provided necessary and sufficient conditions for a connected graph to have a generalized square root [Muk67], whilst Geller in 1970 likewise found necessary and sufficient conditions for a digraph to have a square root [Gel70].

In 1994 Motwani and Sudan established the complexity of the square problem for the generalized undirected case and showed that the problem of determining if a given graph has a square root is a NP-complete problem via a polynomial reduction to 3-CNF Not-All-Equal SAT [MS94].

Their work led to joint work by Farzad, Le, Tuy, Lau and Karimi in 2009 and 2012, in which they established a partial dichotomy theorem for the complexity of determining if a given undirected graph of girth n has a square root [Far+09] which was informally completed under [FK12]. In 2013, Cochefert and others gave an algorithm for constructing a square root with minimal number of edges from undirected graphs with max degree ≤ 6 in polynomial time and an algorithm for constructing maximal edge square roots in exponential time [Coc+13].

In recent years, Duffy provided a full classification for the undirected square of oriented graphs satisfying max degree ≤ 3 and girth ≥ 4 [Duf25].

1.4 Definitions

1.4.1 Complexity Theory

Complexity theory studies the classification of computational problems and explores the amount of resources required to solve certain problems. The critical resource that is measured is running time and is standardized across various different problems by using

Definition 1.4.1 (Big- O Notation). *Let $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. The **Big- O Notation** $f(n) = O(g(n))$ is written if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that for all $n \geq n_0$*

$$f(n) \leq cg(n).$$

Using this notation, we can classify algorithms based on their runtime.

Definition 1.4.2 (Polynomial-Time Algorithm). *An algorithm is said to run in **polynomial time** if there exists a constant $c \geq 0$ such that its run time on inputs of size n is $O(n^c)$.*

Polynomial-time algorithms are important because they are generally regarded as efficient and computationally “simple”. Consequently, the class of problems that admit polynomial-time algorithms forms the basis for the complexity class **P**.

Definition 1.4.3 (Decision Problem). *A **decision problem** is a computational problem whose output is either **Yes** or **No** for every valid input instance.*

Definition 1.4.4 (Class **P**). *The complexity class **P** consists of all decision problems that can be solved by a deterministic algorithm in polynomial time.*

Unfortunately many important decision problems are not known to admit such polynomial-time algorithms and hence need additional classification on their “hardness”. For these problems, despite not have an efficient algorithm for finding a solution, a proposed solution can be verified efficiently.

Definition 1.4.5 (Certificate). *Let L be a decision problem. A **certificate** for an instance x of L is a finite string c that provides evidence that x is a **Yes-instance** of L . A certificate is said to be **polynomially verifiable** if there exists a deterministic algorithm that for given input (x, c) , can determine whether c is a valid certificate for x in time polynomial in $|x|$.*

Definition 1.4.6 (Class **NP**). *The complexity class **NP** consists of all decision problems for which every **Yes-instance** admits a certificate that is polynomially verifiable.*

Since every decision problem in **P** can be solved in polynomial time, a solution can also be verified in polynomial time,

$$\mathbf{P} \subseteq \mathbf{NP}.$$

Whether this inclusion is strict remains one of the most important open questions in theoretical computer science - the **P** vs **NP** problem. As **NP** is a large space, the relative difficulty of problems in **NP** is compared by using reduction.

Definition 1.4.7 (Polynomial-Time Reduction). *Let A and B be decision problems. A **polynomial-time reduction** from A to B , denoted $A \leq_p B$, is a polynomial-time computable function f such that, for every instance x of A ,*

$$x \in A \iff f(x) \in B.$$

Definition 1.4.8 (Class **NP-Hard**). *A decision problem B is **NP-hard** if every problem $A \in \mathbf{NP}$ satisfies $A \leq_p B$. The collection of all **NP-hard** problems is denoted by the class **NP-hard**.*

Definition 1.4.9 (Class **NP-Complete**). *A decision problem B is **NP-complete** if $B \in \mathbf{NP}$, and B is **NP-hard**. The collection of all **NP-complete** problems is denoted by the class **NP-complete**.*

1.4.2 Graph Theory

Definition 1.4.10 (Graph).

Definition 1.4.11 (Undirected Graph).

Definition 1.4.12 (Directed Graph).

Definition 1.4.13 (Graph Distance).

Definition 1.4.14 (Graph Root).

Definition 1.4.15 (Graph Square).

Definition 1.4.16 (Adjacency Matrix).

1.4.3 Mathematical Optimization

Definition 1.4.17 (Integer Program).

Definition 1.4.18 (Linear Program).

Let the *graph root* $H = (V, E_H)$ be a finite, simple, undirected graph, where V is the vertex set with $|V| = n$, and $E_H \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ is the edge set. Denote by $d_H(u, v)$ the *graph distance* defined as the number of edges of the shortest path between vertices u and v in H . If no such path exists, $d_H(u, v) = \infty$.

For vertices $u, v \in V$, the *graph distance* $d_H(u, v)$ is the length of a shortest path between u and v in H . If no such path exists, then $d_H(u, v) = \infty$.

The *square* of the graph H , denoted by $H^2 = (V, E_{H^2})$ is the graph on the same vertex set with edge set

$$E_{H^2} = \{uv \mid u \neq v, 1 \leq d_H(u, v) \leq 2\}. \quad (1.1)$$

That is, two distinct vertices are adjacent in H^2 if and only if they are connected by a path of length one or two in H . This is the standard definition of the square of a graph.

Using an alternative construction, let the *graph square* $G = (V, E_G)$ be an undirected graph defined on the same vertex set V . Let $A_H \in \{0, 1\}^{n \times n}$ denote the adjacency matrix of H , where

$$(A_H)_{ij} = \begin{cases} 1, & \text{if } ij \in E_H, \\ 0, & \text{otherwise,} \end{cases}$$

and let $D_H \in \mathbb{R}^{n \times n}$ denote the degree matrix of H , where

$$(D_H)_{ij} = \begin{cases} \deg_H(i), & \text{if } i = j, \\ 0, & \text{otherwise.} \end{cases}$$

Let $\mathbb{1}_{\geq 1} : \mathbb{R}^{n \times n} \rightarrow \{0, 1\}^{n \times n}$ denote the entrywise *indicator function*, defined by

$$\mathbb{1}_{\geq 1}(M)_{ij} = \begin{cases} 1, & \text{if } M_{ij} \geq 1, \\ 0, & \text{otherwise.} \end{cases}$$

Define the adjacency matrix of G using the adjacency and degree matrices of H

$$A_G = \mathbb{1}_{\geq 1}((A_H)^2 - D_H + A_H). \quad (1.2)$$

Chapter 2

Algorithmic Approaches

In this chapter we explore different methodologies that have been used to show certain properties of the graph root and the graph square. For many restricted cases which are NP, a polynomial time reduction is given and incases which are polynomial, a polynomial time algorithm is given.

We then examine the relaxed linear program which can be evaluated in polynomial time using interior point method, and integrate existing literature on special root structures as a stepping-stone towards generating a true polynomial time algorithm.

2.1 Polynomial Time Algorithm

Ross and Harary give an algorithm for constructing a tree T when given its square graph $G = T^2$ using the following characterization.

- If $G = K_p$ is complete, then T must be a tree of depth 1 (result of Lemma 1)
- If $G \neq K_p$
 - (i) Every vertex of G forms a clique with its neighbours and G is connected (result of Lemma 2)
 - (ii) If two cliques intersect at only one point v then there is a third clique with which they share v and exactly one other point (result of Lemma 3 and Lemma 5)
 - (iii) A clique $C(v)$ in G corresponding to v in T contains exactly as many points in multiple cliques as the number of cliques which include v (result of Lemma 3 and Lemma 4)
 - (iv) Every pair of cliques intersect at at most two points (result of Lemma 5)
 - (v) The number of pairs of cliques that meet at two points is one less than the number of cliques (result of cliques formed from non-leaf nodes forming a tree structure)

Lemma 1

G is complete iff T has depth 1.

Proof. ($\leftarrow$) Let T be a tree with depth 1, hence all vertices in T are at most 2-distance away from any other vertex since the root is a non-leaf vertex adjacent to all vertices in $T \implies G = T^2$ is complete since edges exist between all vertices

($\rightarrow$) Let $G = T^2$ be complete, then all vertices must have distance at most two in T and T has depth at most 2. Since T is a tree, there are no cycles and hence there cannot be depth 2 since

this would lead to a path of length 3 from the vertex in depth 2 to an alternate vertex in depth 1. Hence T has exactly depth 1. $\square$

Lemma 2

Every vertex of G forms a clique with its neighbours and G is connected

Proof. For a vertex $v \in T$, its neighbours in T are all adjacent edge in G by construction hence $\{v\} \cup \Gamma(v)$ is a clique in G . $T \subseteq G$ and since T is a tree which is connected, G must also be connected. $\square$

Lemma 3

Every maximal clique of G corresponds to a closed neighbourhood of a non-leaf vertex and every closed neighbourhood of a non-leaf vertex in T form a clique in G

Proof. A set of leaves in T cannot form a clique in G without having at least one non-leaf since the distance between every leaf is at least greater than 2 unless they share a common neighbour.

Consider a non-leaf vertex $v \in T$, since all vertices in neighbourhood $\Gamma(v)$ have distance at most 2 in G , they will form a clique in G . This clique is maximal since all neighbours of v are already part of the clique and no additional vertex can be added. $\square$

Lemma 4

If G has more than one maximal clique, then vertex v is in exactly one maximal clique in G iff v is a leaf in T

Proof. ($\leftarrow$) Suppose vertices v, w to be in two cliques, there must be a neighbour $x \in \Gamma(v)$ that is not adjacent to w . If x is not a leaf in T , will form a third clique in G and being 2-distance away from w will also be in the clique formed from w . Hence only if this adjacent vertex is a leaf will it be in only one clique G .

($\rightarrow$) Suppose v is a leaf in T whereby v has exactly one neighbour w with is a non-leaf. The only maximal clique in G that includes v must also include w and the neighbourhood $\Gamma(w)$. Since v has no other neighbours it cannot appear in any other clique and hence is in exactly one clique. $\square$

Lemma 5

Two non-leaves of T are adjacent in T iff their neighbourhoods are cliques of G that intersect in exactly two points

Proof. ($\leftarrow$) Supposes cliques in G correspond to the neighbourhoods of two non-leaf neighbours v, w which intersect at exactly v, w . Then v, w must be adjacent, otherwise if they were instead 2-distance away then they would share a common neighbour u which is also in both cliques.

($\rightarrow$) Let v, w be adjacent vertices that are non-leaves in T . Both will form cliques in G and be in each clique since they are neighbours. Assume to the contrary that $\exists$ vertex k also in both cliques. k must be adjacent to both v, w in T but this creates a cycle since there is now two different paths between v, w . Hence v, w are the only two vertices that intersect in the two clique neighbourhoods. $\square$

Lemma 6

G is a connected graph where the removal of any edge does not result in a disconnected graph

Proof. By construction $\exists$ two paths between every pair of vertices $v, w \in G$ where one of the vertices v must be non-leaf. Remove the existing vw edge in $T \subset G$. There still exists a path by taking the 2-distance edge to the neighbour u of v and then going from u to w (vu, uw), hence the graph G is still connected. $\square$

Combining this gives the following

Algorithm

-
- 1: **Input:** A graph G
 - 2: **Output:** A tree T where $G = T^2$
 - Step 1**
 - 3: **if** $G = K_p$ is complete **then**
 - 4: **Return** tree T with depth 1 where root can be any vertex
 - 5: **else**
 - 6: **Continue**
 - 7: **end if**
 - Step 2**
 - 8: Identify all maximal cliques $C_1, \dots, C_n$ of G using an algorithm such as Bron–Kerbosch where $n > 1$ using characterization (i)
 - Step 3**
 - 9: Let $b_1, \dots, b_n$ be multicliqual points corresponding to $C_1, \dots, C_n$ formed from characterization (iii)
 - 10: Construct a graph S with vertex set $\{b_1, \dots, b_n\}$
 - 11: **for** each pair (C_i, C_j) satisfying $|C_i \cap C_j| = 2$ **do**
 - 12: Add edge between b_i and b_j in S
 - 13: **end for**
 - 14: By characterization (v), S is a tree
 - Step 4**
 - 15: **for** each clique C_i **do**
 - 16: Let n_i be the number of unicliqual points in C_i
 - 17: Attach n_i leaf nodes to b_i in S
 - 18: **end for**
 - 19: The resulting tree T has the same number of nodes as G and satisfies $T^2 = G$
-

2.2 NP-Completeness Proofs

2.3 Integer Program Foundation

2.3.1 Background

Lemma 1. *The graph G defined by (1.2) is equivalent to the graph square H^2 defined in (1.1)*

$$ij \in E_{H^2} \iff 1 \leq d_H(i, j) \leq 2. \quad (2.1)$$

Proof. For $i \neq j$, since D_H is diagonal, have $(D_H)_{ij} = 0$ and hence

$$((A_H)^2 - D_H + A_H)_{ij} = (A_H)_{ij}^2 + (A_H)_{ij}.$$

($\Rightarrow$) Suppose $ij \in E_G$ such that $(A_H)_{ij}^2 + (A_H)_{ij} \geq 1$. This implies that at least one of $(A_H)_{ij} = 1$ or $(A_H)_{ij}^2 \geq 1$ must be true. If $(A_H)_{ij} = 1$, then $d_H(i, j) = 1$. If $(A_H)_{ij}^2 \geq 1$, then there exists a vertex k such that $ik, kj \in E_H$, and hence $d_H(i, j) \leq 2$. Therefore $1 \leq d_H(i, j) \leq 2$, and thus $ij \in E_{H^2}$.

($\Leftarrow$) Suppose $ij \in E_{H^2}$ such that $1 \leq d_H(i, j) \leq 2$. D_H is diagonal such that $(D_H)_{ij} = 0$. If $d_H(i, j) = 1$, then $(A_H)_{ij} = 1$. If $d_H(i, j) = 2$, then there exists a vertex k such that $ik, kj \in E_H$, and hence $(A_H)_{ij}^2 \geq 1$. In either case, $(A_H)_{ij}^2 + (A_H)_{ij} \geq 1$, so $(A_G)_{ij} = 1$, and hence $ij \in E_G$.

For $i = j$, since all graphs are simple, have $(E)_{ii} = 0$, $(A)_{ii} = 0$ and hence

$$((A_H)^2 - D_H + A_H)_{ii} = (A_H)_{ii}^2 - (D_H)_{ii}.$$

$(A_H)_{ii}^2 - (D_H)_{ii} = 0$ is always true since the degree matrix of H always has diagonal values $(D_H)_{ii} = \sum_k x_{ik}$ and each edge from vertex i to its neighbour in H will always create a loop to itself in the square such that $(A_H)_{ii}^2 = \sum_{k=1} x_{ik}x_{ki} = \sum_{k=1} x_{ik}^2$ by the symmetry of adjacency matrix A_H . Since $x_{ik} \in \{0, 1\}$, the square of the boolean is identical to the boolean x_{ik} such that $(A_H)_{ii}^2 = \sum_k x_{ik} = (D_H)_{ii}$ and $(A_G)_{ii} = \mathbb{1}_{\geq 1}((A_H)^2 - D_H + A_H)$ as required.

Combining both gives that $E_G = E_{H^2}$, and so $G = H^2$. □

The formulation of the adjacency matrix of the graph square in Equation (1.2) is very powerful. However, the presence of the indicator function introduces a nonlinearity that prevents the problem from being expressed as a purely quadratic form, thereby limiting the direct applicability of existing results from the quadratic optimization literature.

Lemma 2. *If there are no K_3 triangle subgraphs or any C_4 square subgraphs in the graph root such that H has girth of length 5, then Equation (1.2) can be simplified to become*

$$A_G = ((A_H)^2 - D_H + A_H). \quad (2.2)$$

Proof. Since the graph root has girth of length 5 there are no cycles of length 3 or 4 in H . Thus for every pair of vertices $i \neq j \in V$ there is either no cycle that connects i, j or there is a cycle of at least length 5 between them. In all cases $(D_H)_{ij} = 0$. If there is no cycle, then there is at most one path between i, j

i) If the path is of length 1, then $(A_H)_{ij} = 1, (A_H)_{ij}^2 = 0 \implies (A_G)_{ij} = 0 - 0 + 1 = 1$.

- ii) If the path is of length 2, then $(A_H)_{ij} = 0, (A_H)_{ij}^2 = 1 \implies (A_G)_{ij} = 1 - 0 - 0 = 1$.
- iii) If the path is of length ≥ 3 or a path does not exist, then $(A_H)_{ij} = 0, (A_H)_{ij}^2 = 0 \implies (A_G)_{ij} = 0$.

If there is a cycle of length ≥ 5 between i, j such that there are two paths between i, j

- i) If one path is of length 1, then the other is of length ≥ 4 and hence $(A_H)_{ij} = 1, (A_H)_{ij}^2 = 0 \implies (A_G)_{ij} = 0 - 0 + 1 = 1$.
- ii) If one path is of length 2, then the other is of length ≥ 3 and hence $(A_H)_{ij} = 0, (A_H)_{ij}^2 = 1 \implies (A_G)_{ij} = 1 - 0 - 0 = 1$.
- iii) If the paths are both of length ≥ 3 , then $(A_H)_{ij} = 0, (A_H)_{ij}^2 = 0 \implies (A_G)_{ij} = 0$.

If $i = j$, then $(A_H)_{ii} = 0, (D_H)_{ii} = \deg_H(i)$, and $(A_H)_{ii}^2 = \deg_H(i)$. Thus,

$$(A_H)_{ii}^2 - (D_H)_{ii} + (A_H)_{ii} = \deg_H(i) - \deg_H(i) + 0 = 0 = (A_G)_{ii}.$$

□

2.4 Linear Program Relaxation

Bibliography

- [Coc+13] Manfred Cochefert, Jean-François Couturier, Petr A. Golovach, Dieter Kratsch, and Daniël Paulusma. “Sparse Square Roots”. In: *Lecture Notes in Computer Science* 8165 (2013), pp. 177–188.
- [Duf25] Christopher Duffy. “Quasi-Transitive Mixed Graphs and Undirected Squares of Oriented Graphs”. In: *Australasian Journal of Combinatorics* 92.2 (2025), pp. 142–163.
- [Far+09] Babak Farzad, Lap Chi Lau, Van Bang Le, and Nguyen Ngoc Tuy. “Computing Graph Roots Without Short Cycles”. In: *Proceedings of the 26th International Symposium on Theoretical Aspects of Computer Science (STACS 2009)*. Vol. 3. Leibniz International Proceedings in Informatics (LIPIcs). 2009, pp. 397–408.
- [FBB23] Leon Festinger, Françoise Bahoken, and Laurent Beauguitte. “Leon Festinger, 1949, The Analysis of Sociograms Using Matrix Algebra. Version bilingue et commentée”. In: (2023).
- [FK12] Babak Farzad and Majid Karimi. “Square-Root Finding Problem In Graphs, A Complete Dichotomy Theorem”. In: (2012).
- [Fle74] Herbert Fleischner. “The square of every nonseparable graph is Hamiltonian”. In: *Journal of Combinatorial Theory, Series B* 16.1 (1974), pp. 29–34.
- [Gel70] Dennis P. Geller. “The Square Root of a Digraph”. In: *Journal of Combinatorial Theory* 9.1 (1970), pp. 65–68.
- [HKT67] Frank Harary, Richard M. Karp, and William T. Tutte. “A criterion for planarity of the square of a graph”. In: *Journal of Combinatorial Theory* 2 (1967), pp. 406–410.
- [Lau06] Lap Chi Lau. “Bipartite Roots of Graphs”. In: *ACM Transactions on Algorithms* 2.2 (2006), pp. 178–208.
- [LC04] Lap Chi Lau and Derek G. Corneil. “Recognizing Powers of Proper Interval, Split, and Chordal Graphs”. In: *SIAM Journal on Discrete Mathematics* 18.1 (2004), pp. 83–102.
- [LT07] Van Bang Le and Nguyen Ngoc Tuy. “The square of a block graph”. In: *Discrete Mathematics* 307.21 (2007), pp. 2706–2715.
- [MS94] Rajeev Motwani and Madhu Sudan. “Computing roots of graphs is hard”. In: *Discrete Applied Mathematics* 54.1 (1994), pp. 81–88.
- [Muk67] A. Mukhopadhyay. “The Square Root of a Graph”. In: *Journal of Combinatorial Theory* 2 (1967), pp. 290–295.
- [PA23] Or Peres and C. Avin. “Distributed Demand-aware Network Design using Bounded Square Root of Graphs”. In: *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications* (2023), pp. 1–10.

- [RH60] Ian C. Ross and Frank Harary. “The square of a tree”. In: *The Bell System Technical Journal* 39.3 (1960), pp. 641–647.